

```
!pip install opendatasets
```

```
Collecting opendatasets
  Downloading opendatasets-0.1.22-py3-none-any.whl.metadata (9.2 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from opendatasets) (4.67.1)
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (from opendatasets) (1.7.4.5)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from opendatasets) (8.3.1)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (6.3.0)
Requirement already satisfied: certifi>=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (2024.7.4)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (3.3.2)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (5.29.5)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (2.9.0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (75.1.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (1.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle->opendatasets) (0.5.1)
Downloading opendatasets-0.1.22-py3-none-any.whl (15 kB)
Installing collected packages: opendatasets
Successfully installed opendatasets-0.1.22
```

```
import opendatasets as od
```

```
od.download("https://www.kaggle.com/datasets/ryanbadai/clothes-dataset")
```

```
Dataset URL: https://www.kaggle.com/datasets/ryanbadai/clothes-dataset
Downloading clothes-dataset.zip to ./clothes-dataset
100%|██████████| 1.37G/1.37G [00:14<00:00, 99.3MB/s]
```

```
DATASET_PATH = "/content/clothes-dataset/Clothes_Dataset"
```

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
```

```
transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.5,0.5,0.5],[0.5,0.5,0.5])
])

dataset = datasets.ImageFolder(DATASET_PATH, transform=transform)

num_classes = len(dataset.classes)
print("Classes:", dataset.classes)
```

```
Classes: ['Blazer', 'Celana_Panjang', 'Celana_Pendek', 'Gaun', 'Hoodie', 'Jaket', 'Jaket_Denim', 'Jaket_Olahraga', 'Jeans',
```

```
train_size = int(0.7 * len(dataset))
val_size   = int(0.15 * len(dataset))
test_size  = len(dataset) - train_size - val_size

train_ds, val_ds, test_ds = random_split(
    dataset, [train_size, val_size, test_size]
)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_ds, batch_size=32)
test_loader  = DataLoader(test_ds, batch_size=32)
```

```
class DeepCNN(nn.Module):

    def __init__(self, num_classes):
```

```

super(DeepCNN, self).__init__()

self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
self.conv3 = nn.Conv2d(64, 128, 3, padding=1)

self.pool = nn.MaxPool2d(2,2)

self.fc1 = nn.Linear(128*16*16, 256)
self.fc2 = nn.Linear(256, num_classes)

self.dropout = nn.Dropout(0.5)

def forward(self, x):

    x = self.pool(torch.relu(self.conv1(x)))
    x = self.pool(torch.relu(self.conv2(x)))
    x = self.pool(torch.relu(self.conv3(x)))

    x = x.view(x.size(0), -1)

    x = self.dropout(torch.relu(self.fc1(x)))

    x = self.fc2(x)

    return x

```

```

class ShallowCNN(nn.Module):

    def __init__(self, num_classes):
        super(ShallowCNN, self).__init__()

        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)

        self.pool = nn.MaxPool2d(2,2)

        self.fc1 = nn.Linear(64*32*32, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):

        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))

        x = x.view(x.size(0), -1)

        x = torch.relu(self.fc1(x))

        x = self.fc2(x)

        return x

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

def train_model(model, train_loader, val_loader, epochs=10):

    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_acc_list = []
    val_acc_list = []

    for epoch in range(epochs):

        # Training
        model.train()
        correct, total = 0, 0

        for images, labels in train_loader:

            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

```

```

        outputs = model(images)

        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        preds = outputs.argmax(1)

        correct += (preds == labels).sum().item()
        total += labels.size(0)

    train_acc = correct / total
    train_acc_list.append(train_acc)

    # Validation
    model.eval()
    correct, total = 0, 0

    with torch.no_grad():

        for images, labels in val_loader:

            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            preds = outputs.argmax(1)

            correct += (preds == labels).sum().item()
            total += labels.size(0)

    val_acc = correct / total
    val_acc_list.append(val_acc)

    print(f"Epoch {epoch+1}/{epochs}")
    print(f"Train Acc: {train_acc:.4f} | Val Acc: {val_acc:.4f}")
    print("-"*40)

    return train_acc_list, val_acc_list

```

Device: cuda

```

def test_model(model, test_loader):

    model.eval()

    correct, total = 0, 0

    with torch.no_grad():

        for images, labels in test_loader:

            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            preds = outputs.argmax(1)

            correct += (preds == labels).sum().item()
            total += labels.size(0)

    return correct / total

```

```

deep_model = DeepCNN(num_classes)

deep_train_acc, deep_val_acc = train_model(
    deep_model,
    train_loader,
    val_loader,
    epochs=10
)

deep_test_acc = test_model(deep_model, test_loader)

```

```
print("Deep CNN Test Accuracy:", deep_test_acc)
```

```
Epoch 1/10
Train Acc: 0.1747 | Val Acc: 0.3280
-----
Epoch 2/10
Train Acc: 0.3267 | Val Acc: 0.4062
-----
Epoch 3/10
Train Acc: 0.4036 | Val Acc: 0.4818
-----
Epoch 4/10
Train Acc: 0.4632 | Val Acc: 0.5076
-----
Epoch 5/10
Train Acc: 0.5086 | Val Acc: 0.5129
-----
Epoch 6/10
Train Acc: 0.5610 | Val Acc: 0.5440
-----
Epoch 7/10
Train Acc: 0.5874 | Val Acc: 0.5724
-----
Epoch 8/10
Train Acc: 0.6303 | Val Acc: 0.5644
-----
Epoch 9/10
Train Acc: 0.6730 | Val Acc: 0.5636
-----
Epoch 10/10
Train Acc: 0.6947 | Val Acc: 0.5662
-----
Deep CNN Test Accuracy: 0.5653333333333334
```

```
shallow_model = ShallowCNN(num_classes)

shallow_train_acc, shallow_val_acc = train_model(
    shallow_model,
    train_loader,
    val_loader,
    epochs=10
)

shallow_test_acc = test_model(shallow_model, test_loader)

print("Shallow CNN Test Accuracy:", shallow_test_acc)
```

```
Epoch 1/10
Train Acc: 0.2048 | Val Acc: 0.3316
-----
Epoch 2/10
Train Acc: 0.4280 | Val Acc: 0.4151
-----
Epoch 3/10
Train Acc: 0.5354 | Val Acc: 0.4684
-----
Epoch 4/10
Train Acc: 0.6230 | Val Acc: 0.4791
-----
Epoch 5/10
Train Acc: 0.7002 | Val Acc: 0.4782
-----
Epoch 6/10
Train Acc: 0.7731 | Val Acc: 0.4800
-----
Epoch 7/10
Train Acc: 0.8478 | Val Acc: 0.4853
-----
Epoch 8/10
Train Acc: 0.8910 | Val Acc: 0.4996
-----
Epoch 9/10
Train Acc: 0.9114 | Val Acc: 0.4933
-----
Epoch 10/10
Train Acc: 0.9326 | Val Acc: 0.4862
-----
Shallow CNN Test Accuracy: 0.4968888888888889
```

```
print(" MODEL COMPARISON ")

print(f"Deep CNN:")
print(f" Convolution Layers : 3")
print(f" Final Val Accuracy : {deep_val_acc[-1]*100:.2f}%")
print(f" Test Accuracy : {deep_test_acc*100:.2f}%")
```

```
print(f"Shallow CNN:")
print(f"  Convolution Layers : 2")
print(f"  Final Val Accuracy : {shallow_val_acc[-1]*100:.2f}%")
print(f"  Test Accuracy      : {shallow_test_acc*100:.2f}%")
```

MODEL COMPARISON

Deep CNN:

```
Convolution Layers : 3
Final Val Accuracy : 56.62%
Test Accuracy      : 56.53%
```

Shallow CNN:

```
Convolution Layers : 2
Final Val Accuracy : 48.62%
Test Accuracy      : 49.69%
```

Start coding or [generate](#) with AI.